Task-18:- Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements for deploying multiple demo websites on AWS infrastructure using Nginx, Application Load Balancer (ALB), SSL certificates, and Route 53 DNS.

This project demonstrates how multiple websites can be hosted and accessed through secure and scalable cloud architecture.

The intended audience of this document includes:

- DevOps Engineers
- Cloud Engineers
- System Administrators
- Project Reviewers
- Technical Team Members

This document provides a structured description of the project requirements, system behavior, and technical constraints.

1.2 Scope

The scope of this project includes deploying four websites on AWS infrastructure with proper DNS routing and secure HTTPS access.

The implementation includes:

Included in Scope

- Deploying EC2 instances on AWS
- Installing and configuring **Nginx**
- Hosting **two websites directly on an EC2 instance**
- Deploying **two additional EC2 instances**
- Creating an **Application Load Balancer**
- Configuring **target groups and health checks**
- Setting up **SSL certificates using AWS Certificate Manager**
- Configuring **HTTPS access**
- Creating **DNS records using Route 53**
- Testing and validating website accessibility

Not Included in Scope

- CI/CD pipeline implementation
- Auto Scaling configuration
- Containerization using Docker
- Kubernetes deployment
- Production-grade monitoring systems
- Database integration

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
AWS	Amazon Web Services
EC2	Elastic Compute Cloud – Virtual Machine service

Nginx	High-performance web server and reverse proxy
ALB	Application Load Balancer
DNS	Domain Name System
Route 53	AWS managed DNS service
ACM	AWS Certificate Manager
SSL	Secure Socket Layer
HTTPS	Secure HTTP protocol
Target Group	Logical group of instances receiving traffic from ALB

1.4 References

The following references were used during the preparation of this project:

- GitHub Repository
  GitHub - pushkar-iamops/hello-world-flask
- AWS EC2 Documentation
  https://docs.aws.amazon.com/ec2/
- AWS Load Balancer Documentation
  https://docs.aws.amazon.com/elasticloadbalancing/
- AWS Route 53 Documentation
  https://docs.aws.amazon.com/route53/
- AWS Certificate Manager Documentation
  https://docs.aws.amazon.com/acm/

1.5 Overview

This document provides a complete requirement analysis for deploying multiple websites using AWS cloud services.

The document is structured as follows:

1. **Introduction** – Overview of the project purpose and scope
2. **Project Overview** – Objectives, business goals, and constraints
3. **Functional Requirements** – System functionalities and use cases
4. **Non-Functional Requirements** – Security, performance, reliability requirements
5. **Assumptions and Dependencies** – Project assumptions and dependencies
6. **Risk Analysis** – Potential risks and mitigation strategies
7. **Glossary** – Explanation of key terms used in the document
8. **Appendices** – Additional supporting information
9. **Customer Acknowledgement** – Confirmation from stakeholders

---

2. Project Overview

2.1 Project Objectives

The main objectives of this project are:

- Deploy multiple websites on AWS infrastructure
- Configure **Nginx virtual hosts** to serve multiple sites

- Implement **Application Load Balancer** for traffic distribution
- Enable **secure HTTPS communication using SSL**
- Configure **DNS routing using Route 53**
- Demonstrate a scalable cloud-based architecture
- Validate website accessibility and load balancing functionality

---

2.2 Business Goals

The project aims to achieve the following business goals:

- Demonstrate scalable **cloud web hosting architecture**
- Improve **traffic distribution using load balancing**
- Provide **secure communication through HTTPS**
- Simplify **domain management using Route 53**
- Allow testing of internal demo websites
- Enable learning and implementation of **DevOps infrastructure practices**

2.3 Background

ABC Technologies wants to host multiple internal demo websites on AWS for testing cloud-based hosting and load balancing.

The company requires a system where:

- Some websites are hosted directly on a **single Nginx server**
- Other websites are served through a **load balancer distributing traffic across multiple servers**

Each website will be based on the same **Hello World Flask application** from a GitHub repository, but the page content will be modified so that each site displays a unique message.

This setup helps demonstrate real-world **DevOps deployment architecture using AWS services**.

2.4 Constraints

The project must operate under the following constraints:

Time Constraint

The task must be completed within **3 hours**.

Technology Constraint

The system must use the following technologies:

- AWS EC2
- Nginx
- Application Load Balancer
- AWS Route 53
- AWS Certificate Manager

Budget Constraint

The infrastructure must use minimal AWS resources to reduce cost.

Resource Constraint

Limited number of EC2 instances should be used.

---

3. Functional Requirements

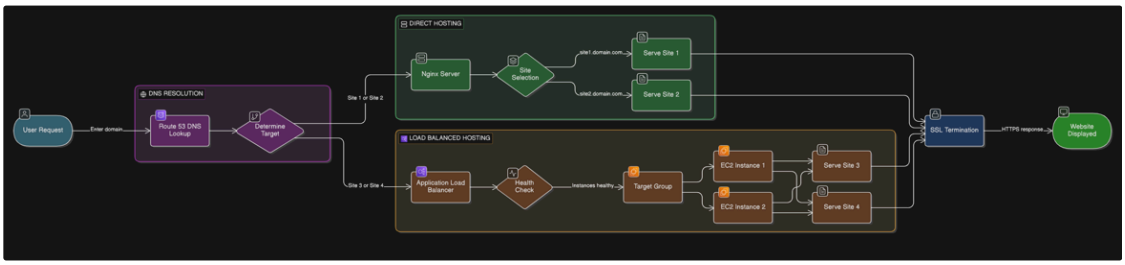
3.1 Use Case Diagram

The use case diagram represents the interaction between the **User** and the deployed web services.

Actor: **User**

Use cases:

- Access Site 1
- Access Site 2
- Access Site 3
- Access Site 4
- Resolve DNS via Route 53
- Route traffic through Load Balancer
- Secure connection using HTTPS



3.2 Use Cases

3.2.1 Use Case 1: Access Site 1

Description

User accesses the first website hosted directly on the Nginx server.

Actors

User

Preconditions

- EC2 instance with Nginx must be running
- DNS record must exist in Route 53
- Website must be configured in Nginx server block

Postconditions

The browser displays the message:

Hello from Site 1

Flow of Events

1. User enters site1.domain.com in the browser
2. Route 53 resolves the domain to the EC2 instance
3. Request reaches the Nginx server
4. Nginx serves the website content
5. Browser displays the website

Alternative Flows

Alternative Flow 1

If DNS is not configured correctly, the site will not load.

Alternative Flow 2

If the Nginx service is stopped, the request will fail.

3.2.2 Use Case 2: Access Site 2

Description

User accesses the second website hosted on the same Nginx server.

Actors

User

Preconditions

- Nginx server block must be configured for Site 2
- DNS record must exist

Postconditions

The browser displays:

Hello from Site 2

Flow of Events

1. User enters site2.domain.com
2. Route 53 resolves DNS
3. Request reaches Nginx server
4. Nginx identifies the server block
5. Correct website content is displayed

3.2.3 Use Case 3: Access Site 3

Description

User accesses the third website hosted on EC2 instances behind a load balancer.

Actors

User

Preconditions

- Load balancer must be running
- Target group must contain EC2 instances
- Health checks must be successful

Postconditions

The website displays:

Hello from Site 3

Flow of Events

1. User enters site3.domain.com
2. Route 53 routes request to ALB
3. ALB receives the request
4. ALB forwards traffic to an EC2 instance
5. The website content is returned to the user

3.2.4 Use Case 4: Access Site 4

Description

User accesses the fourth website through the load balancer.

Actors

User

Preconditions

- EC2 instances must be healthy
- Load balancer must be configured

Postconditions

The browser displays:

Hello from Site 4

Flow of Events

1. User opens site4.domain.com
2. Route 53 resolves DNS
3. Request goes to ALB
4. ALB distributes traffic to one instance
5. Website response is returned

---

4. Non-Functional Requirements

4.1 Performance Requirements

- Websites should load within **2–3 seconds**
- Load balancer should distribute traffic efficiently
- Nginx must handle multiple virtual hosts simultaneously

4.2 Security Requirements

- All websites must support **HTTPS**
- SSL certificates must be managed using **AWS Certificate Manager**
- Security groups must allow only required ports:
  - 22 (SSH)
  - 80 (HTTP)
  - 443 (HTTPS)

4.3 Usability Requirements

- Websites must be easily accessible through domain names
- The web interface should clearly display the site identity
- Users should not need any login or authentication

4.4 Reliability Requirements

- Load balancer must remove unhealthy instances automatically
- Health checks must ensure system availability
- Websites should remain accessible even if one server fails

4.5 Compliance Requirements

The system must comply with standard cloud infrastructure practices including:

- Secure HTTPS communication
- DNS best practices
- AWS infrastructure configuration guidelines

---

5. Assumptions and Dependencies

5.1 Assumptions

- AWS account is already available
- Domain name is registered
- GitHub repository is publicly accessible
- Internet connectivity is available

5.2 Dependencies

The project depends on:

- AWS EC2 availability
- AWS Route 53 DNS service
- AWS Certificate Manager
- Internet connectivity
- GitHub repository access

---

6. Risk Analysis

6.1 Risk Identification

Risk	Description
DNS Misconfiguration	Domain may not resolve correctly
Instance Failure	One EC2 instance may stop working
SSL Misconfiguration	HTTPS may not work properly
Health Check Failure	Load balancer may mark instances unhealthy

---

6.2 Risk Mitigation

Risk	Mitigation Strategy
DNS Issues	Verify DNS records and propagation
Instance Failure	Use load balancing
SSL Issues	Validate certificates in ACM
Health Check Errors	Configure correct health check paths

---

7. Glossary

Term	Definition
Load Balancer	Distributes traffic across servers
Virtual Host	Hosting multiple websites on a single server
Health Check	Monitoring server availability
SSL Certificate	Encryption certificate for secure connections

---

8. Appendices

Additional materials that may be included:

- Architecture Diagram
- AWS Console Screenshots
- Nginx Configuration Files
- Route 53 DNS Records
- Load Balancer Target Groups

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____